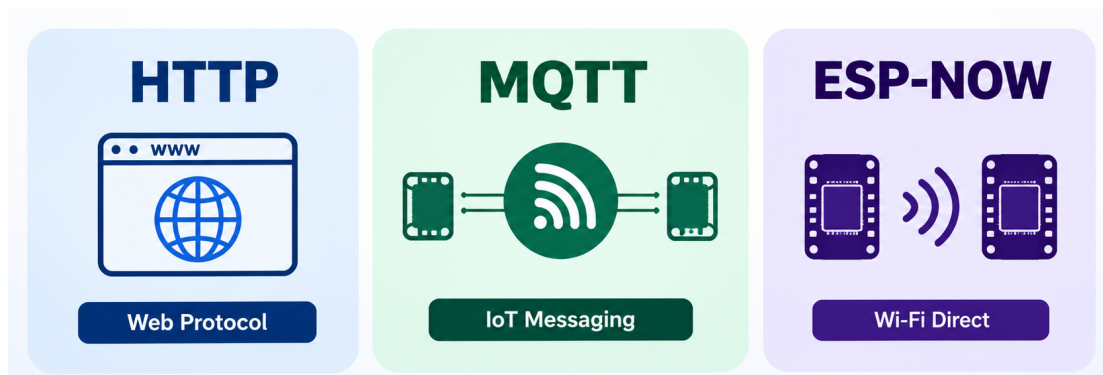


Research Review & Protocol Comparison

Task 2.5 – MQTT vs. ESP-NOW vs. HTTP



*Dana Natsheh
Cyber Robot
IoT*

1. Executive Summary

This report is a catch-up and research-review deliverable rather than a new build: it revisits the last three field-training projects – the ESP-NOW two-way interlock (Task 2.4a), its MQTT re-implementation (Task 2.4b), and the BME280 environmental monitor built around a local HTTP dashboard (Task 2.3) – alongside the ESP32 architecture research report (Task 2.1), and digs into the specific networking concepts that those reports used correctly in practice but never fully unpacked in theory. Three topics were identified as worth researching in depth: what actually happens on the air when an ESP-NOW packet is sent (Section 2.1), the reliability and state-tracking machinery MQTT offers beyond the basic publish/subscribe pattern already used in Task 2.4b (Section 2.2), and why HTTP’s request-response, stateless design behaves so differently from the other two once the connection is examined at the transport level (Section 2.3). A fourth topic, orthogonal to protocol choice, is also researched: how firmware itself – independent of any hardware change – can be written to minimize a device’s power consumption, covering ESP32 sleep modes, Dynamic Frequency Scaling, and a set of general low-power coding habits (Section 2.4). Section 3 consolidates the protocol research, together with what was already demonstrated hands-on, into a single comprehensive comparison table across the three protocols. Section 4 maps the comparison back onto the three completed builds to justify, in hindsight, why each protocol was actually a reasonable fit for its project. Section 5 closes with a summary of what this review changes about protocol selection and firmware design going forward.

2. Topics Revisited in Depth

2.1 What an ESP-NOW Packet Actually Is

Task 2.4a’s interlock treated `esp_now_send()` and `OnDataRecv()` as a reliable-enough black box, since the project’s own fail-safe timeout was designed to tolerate occasional loss anyway. Digging into how ESP-NOW is actually implemented shows why that design choice was the right call, and not just a convenient one.

It is not WiFi, and it is not IP. ESP-NOW is a connectionless protocol defined by Espressif that encapsulates application data directly inside a vendor-specific *IEEE 802.11 action frame* and transmits it from one WiFi radio to another without ever forming a WiFi association, without DHCP, and without an IP address on either side. This is precisely why Task 2.4a’s boards could call `WIFI_STA` and skip `WiFi.begin()` entirely – ESP-NOW only needs the radio to be in a WiFi-capable mode, not actually joined to a network. Both peers must simply agree on the same 2.4 GHz channel.

Payload size is bounded by the 802.11 frame format itself, not by a firmware choice. ESP-NOW v1.0 packets are capped at 250 bytes of payload, because the vendor-specific information element that carries the data uses a single-byte length field in the 802.11 standard (`0xFF` = 255, minus header overhead). ESP-NOW v2.0, introduced in newer ESP-IDF releases, lifts this to roughly 1450 bytes by fragmenting across multiple elements, though the classic 250-byte limit is still the one most Arduino-core projects (including Task 2.4a’s 6-byte `message` struct) are built against, since it comfortably exceeds it either way.

Delivery is acknowledged only at the MAC layer, not at the application layer. The `esp_now_register_send_cb()` callback reports `ESP_NOW_SEND_SUCCESS` only when the frame was received at the WiFi MAC layer of the peer – it says nothing about whether the receiving sketch’s callback ran or whether the data was semantically valid. This is exactly why Task 2.4a’s report was correct to note, in its Known Limitations, that `OnDataSent` did not check status: the protocol itself offers no stronger guarantee to check against, which is also why the project’s own silence-based fail-safe (declaring the link lost after 2 seconds, rather than trusting any single acknowledgement) is the architecturally appropriate way to build reliability on top of ESP-NOW, instead of inside it.

Security is opt-in and peer-limited. Encrypted ESP-NOW peers use CCMP (the same AES-CTR-with-CBC-MAC construction used by WPA2) but the number of simultaneously encrypted peers is capped by the ESP-IDF WiFi driver – historically 6, and configurable up to 17 on recent IDF versions – while unencrypted peers (as used, unencrypted, in Task 2.4a) can number up to 20, or be unlimited via broadcast. This is the concrete mechanism behind the Known Limitations note in Task 2.4a that `peerInfo.encrypt = false` would need revisiting outside a trusted classroom setting: enabling it is not just a boolean flip, it is a real cryptographic handshake with its own peer-count ceiling.

Interesting detail found during research: because ESP-NOW rides on raw 802.11 action frames instead of an IP stack, it has no concept of routing at all – a packet either reaches a radio within range on the same channel, or it does not. There is no ESP-NOW equivalent of a hop, a gateway, or a broker to relay a message further. This is the direct technical reason Task 2.4b existed: to reach a partner board that might not even be on the same WiFi network, the project had no choice but to switch to a protocol built on top of IP – MQTT.

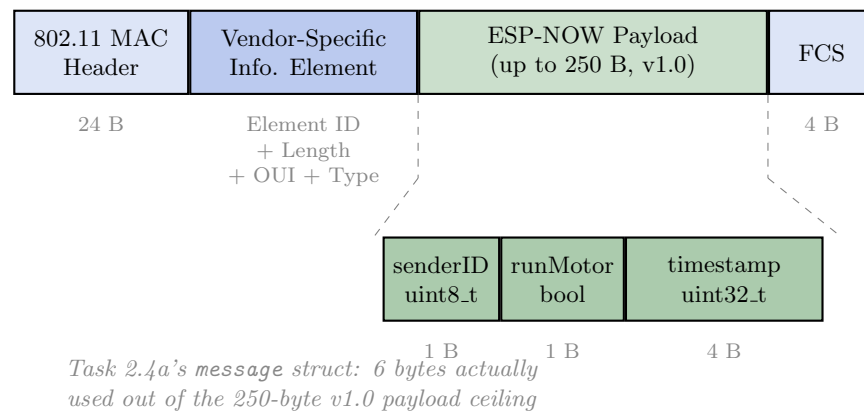


Figure 1: Structure of an ESP-NOW action frame, with Task 2.4a’s 6-byte payload struct shown against the 250-byte v1.0 payload ceiling.

2.2 MQTT Beyond Publish/Subscribe

Task 2.4b implemented the core MQTT loop – connect, subscribe to the partner’s topic, publish to its own topic, reconnect on drop – but deliberately left several protocol-native reliability features unused, noting them explicitly under Future Improvements (Last Will and Testament, QoS 1). Researching what those features actually do clarifies why they were reasonable to defer, and what they would have replaced.

QoS is a delivery contract layered on top of TCP, not a substitute for it. MQTT defines three Quality-of-Service levels: QoS 0 (*at most once*, fire-and-forget, no acknowledgement – what `PubSubClient` used in Task 2.4b defaults to), QoS 1 (*at least once*, acknowledged with a PUBACK, but duplicates are possible if the acknowledgement itself is lost), and QoS 2 (*exactly once*, a four-way

PUBREC/PUBREL/PUBCOMP handshake that eliminates duplicates at the cost of extra round-trips). Because Task 2.4b already published continuously every 200 ms and derived safety purely from elapsed silence rather than from any single message's content, an occasionally dropped QoS 0 packet was never actually a correctness risk – the same reasoning the project used to justify not checking `OnDataSent` status in the ESP-NOW version applies here for exactly the same underlying reason.

Retained messages solve the “What was the last state?” problem that Task 2.4b's arrive_time-based fail-safe deliberately avoids relying on. When a publisher sets the `retain` flag, the broker stores that single message against the topic indefinitely and immediately delivers it to any client that subscribes later, even long after the original publisher disconnected. This is useful for exposing a device's last-known state to a dashboard that connects after the fact, but it is a poor fit for Task 2.4b's actual requirement: a retained “motor should run” message would let a newly (re)connecting board briefly trust a value that could be arbitrarily old, which is precisely the stale-data failure mode the fail-safe timeout exists to prevent. Retained messages and a liveness timeout solve two different problems and are not interchangeable.

Last Will and Testament (LWT) is the broker announcing a disconnect on the client's behalf. A client registers a will message – topic, payload, QoS, and retain flag – at connect time; if the broker detects the client vanished without a clean disconnect (its TCP session simply dies), the broker publishes that will message itself. Applied to Task 2.4b, each board could register an LWT of `runMotor=false` on its own topic, which would let the fail-safe fire the instant the broker notices the disconnect rather than only after the full 2-second silence window – the improvement the project's own report already anticipates in its Future Improvements section, now with the underlying mechanism identified.

Persistent sessions and retained messages are frequently conflated but are stored differently. A persistent session is keyed by client ID on the broker and holds that client's subscriptions plus any undelivered QoS 1/2 messages while it is offline; a retained message is keyed by topic and is completely independent of any specific client's connection state. Task 2.4b's boards reconnect with a freshly-generated client ID derived from the chip's efuse MAC on every boot, which means they never benefit from session persistence in the first place – a deliberate simplicity trade-off worth naming explicitly, since it is different from simply “not using QoS.”

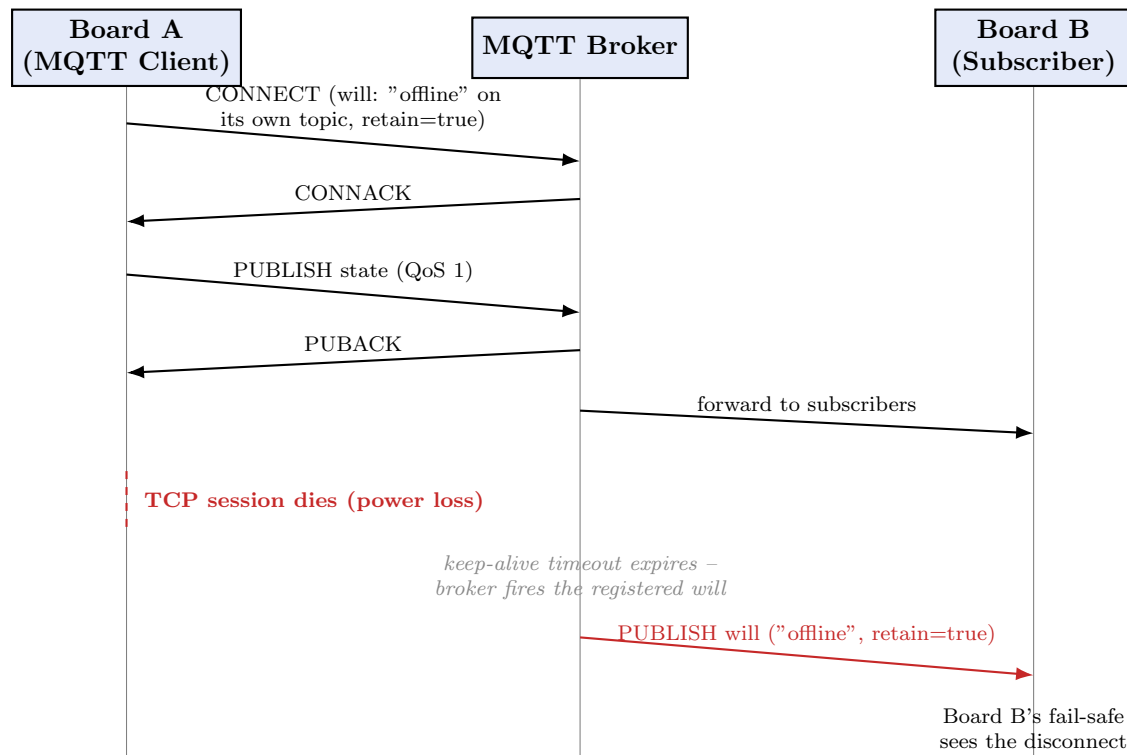


Figure 2: MQTT sequence showing a client connecting with a registered Last Will and Testament, a normal QoS 1 publish/acknowledge cycle, and the broker firing the will message once it detects an ungraceful disconnect.

2.3 Why HTTP Behaves Differently Once You Look at the Connection Itself

Task 2.3's dashboard used `ESPAsyncWebServer` specifically because a synchronous HTTP server would block on a large CSV download; the report reasoned about this correctly at the level of "one request can be slow." The deeper research question is why HTTP's request-response model produces that problem shape in the first place, and how it fundamentally differs from the other two protocols in this report rather than merely being a slower version of the same thing.

HTTP is stateless by design. Each HTTP request carries no memory of any previous request on its own; the server that handled the browser's `GET /data` poll a second ago has, at the protocol level, no inherent notion that the same browser asked `GET /` a moment before. Task 2.3's dashboard works around this by having the browser itself re-poll `/data` on a timer – the client, not the protocol, is what creates the illusion of a live-updating page. This is the opposite of MQTT and ESP-NOW, where the server (broker or peer) can push data to a client at any time without being asked.

Every request re-pays connection overhead unless the connection is deliberately kept alive. An HTTP request over a fresh TCP connection carries the cost of a TCP three-way handshake before a single byte of HTTP is exchanged, whereas MQTT opens one TCP connection and keeps it alive for the duration of the session, amortizing that setup cost across every subsequent publish. This is the concrete, measurable reason MQTT is consistently reported as more bandwidth- and power-efficient than HTTP for frequent small updates – published comparisons put a bare MQTT packet at as little as two bytes of overhead against roughly eight or more for a minimal HTTP request, a gap that compounds every time Task 2.3's dashboard polls `/data`.

HTTP's overhead is the trade for using existing, universal infrastructure. HTTP runs over the exact same TCP/IP stack, the same port-80/443 conventions, and the same browser and firewall support that the entire web already relies on, which is exactly why Task 2.3's dashboard needed no

app, no broker, and no pairing step – any device with a browser on the same network could load it immediately. That universality is bought with per-request verbosity that a purpose-built IoT protocol does not need to pay.

Interesting detail found during research: published power-consumption studies comparing the two protocols on the same hardware found MQTT at QoS 0 and QoS 1 consuming measurably less energy than HTTP polling over long-running tests on a battery-powered microcontroller – a quantified version of exactly the trade-off Task 2.3’s own report reasons about qualitatively when it chose an async server specifically to avoid stalling the poll loop.

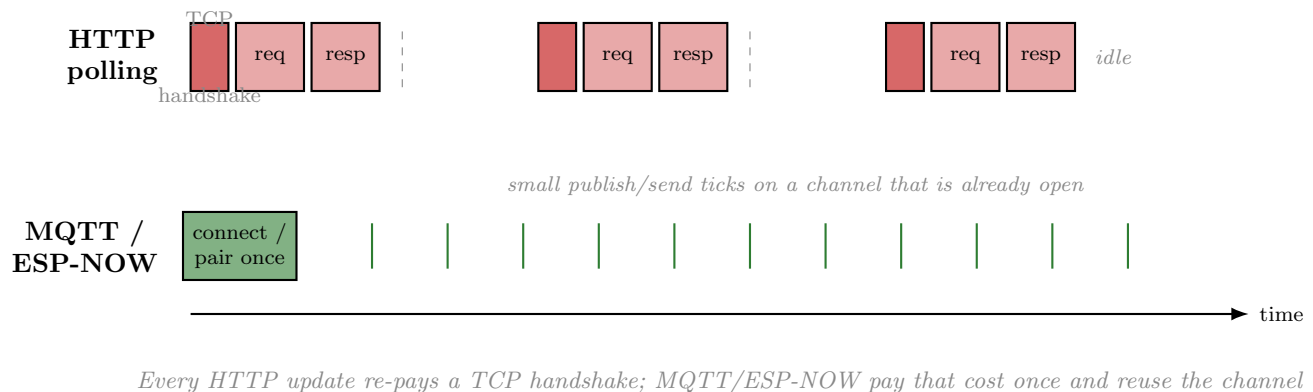


Figure 3: Timeline comparing HTTP’s repeated handshake-plus-request-response cycle against a single long-lived MQTT/ESP-NOW channel carrying the same stream of updates.

2.4 Writing Power-Efficient Code: A Software-Only Approach

None of the four documents reviewed treated power consumption as something the *code itself* controls – Task 2.3’s report measured power only as an outcome to compare protocols by (Section 2.3), and none of the four builds ran on a battery. This subsection researches the software-only levers available on the ESP32: techniques that reduce current draw purely through how the firmware is written, with no change to the board, the power supply, or the surrounding circuit.

The single biggest lever is not running the CPU at all. The ESP32 offers three progressively more aggressive low-power states, all entered and exited entirely from code. Modem-sleep powers down only the WiFi/Bluetooth radio between beacon intervals while the CPU keeps running – it is actually the *default* state once station mode connects, and is controlled with a single API call. Light-sleep additionally clock-gates the CPU and most RAM while fully retaining program state, resuming execution on the very next line after the sleep call with sub-millisecond wake latency. Deep-sleep goes further still, powering down the CPU cores and system RAM entirely and restarting the whole program from `setup()` on wake, retaining only a small block of RTC memory across the reset.

```

1 #include "esp_wifi.h"
2
3 void setup() {
4     WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
5     while (WiFi.status() != WL_CONNECTED) { delay(200); }
6
7     // Radio wakes only on each AP beacon (DTIM interval) instead of
8     // staying powered continuously -- CPU keeps running normally.
9     esp_wifi_set_ps(WIFI_PS_MIN_MODEM);
10 }
```

Listing 1: Modem-sleep, requested once after WiFi connects – keeps a live MQTT session (as in Task 2.4b) while letting the radio sleep between beacons

```

1 RTC_DATA_ATTR int bootCount = 0;    // survives deep sleep, lost on power-on reset
2
3 void setup() {
4     bootCount++;                    // no need to re-fetch this from flash/NVS
5     readSensorAndPublish();
6
7     esp_sleep_enable_timer_wakeup(60ULL * 1000000ULL); // wake in 60s
8     esp_deep_sleep_start();         // execution restarts at setup() on wake
9 }

```

Listing 2: Deep-sleep between readings, with RTC memory used to skip redundant work on every wake – the code-only equivalent of Task 2.3’s NVS persistence, but for the whole device rather than one value

Dynamic Frequency Scaling (DFS) is the code-only middle ground for tasks that cannot sleep at all. Where a project’s main loop must stay resident (as in Task 2.3’s async HTTP server or Task 2.4b’s non-blocking MQTT loop), `esp_pm_configure()` lets the CPU run at its full configured frequency only while a component actively holds a power-management lock, and drop to a much lower frequency the instant every task is idle – entirely automatic once configured, and requiring no change to the surrounding application logic:

```

1 #include "esp_pm.h"
2
3 esp_pm_config_t pmConfig = {
4     .max_freq_mhz = 240,           // used while a task/driver holds the lock
5     .min_freq_mhz = 40,            // used automatically once the CPU is idle
6     .light_sleep_enable = true     // also auto-enter light-sleep when idle
7 };
8 esp_pm_configure(&pmConfig);

```

Listing 3: Dynamic Frequency Scaling – full speed only when something is actually busy

Away from the sleep APIs, several ordinary coding habits compound into real savings. None of these require a sleep call at all:

- **Never busy-wait or poll on a fixed delay().** Every one of the four reviewed projects already follows this instinct correctly – Task 2.3 and Task 2.4b’s non-blocking `millis()`-timed loops keep the CPU free to idle between checks instead of spinning; the same discipline is also what lets the idle task hand control to a low-power wait-for-interrupt state, and what makes DFS and auto light-sleep effective in the first place.
- **Minimize flash/NVS writes, not just for wear (Task 2.3, Section 5.3) but for power.** Every flash erase/write is a real current spike lasting milliseconds; code that only persists a value when it has actually changed, rather than on a fixed timer regardless of content, avoids paying that cost for no new information.
- **Disable peripherals and interfaces the current code path is not using.** A Serial UART left initialized, an ADC left attached, or a driven-but-unused GPIO all draw current for no functional benefit; explicitly powering down or not initializing what a given build cycle doesn’t need is a zero-hardware-cost saving.

- **Prefer integer/fixed-point arithmetic over floating point in hot paths.** The ESP32's FPU still costs cycles (and therefore time-at-full-frequency) per operation; in a high-frequency loop, replacing repeated floating-point math with integer math shortens the time the CPU must stay at a high DFS frequency to finish the work.
- **Batch and duty-cycle radio use instead of transmitting continuously.** Task 2.4a and 2.4b's continuous heartbeat-style broadcast is the right call for a safety link (Section 2.1– 2.2), but for a project without that requirement, coalescing several readings into one transmission and duty-cycling the radio (matching the modem-sleep interval above) avoids paying the WiFi/ESP-NOW radio's per-transmission energy cost once per data point.

Interesting detail found during research: modem-sleep is not an opt-in feature that has to be added to a project – ESP-IDF station mode enables `WIFI_PS_MIN_MODEM` by default the moment WiFi connects. Task 2.4b's firmware, without ever calling `esp_wifi_set_ps()` itself, was therefore already benefiting from modem-sleep between beacon intervals; the code-only lever available on top of that default is choosing `WIFI_PS_MAX_MODEM` with a longer `listen_interval` for further savings, at the cost of higher and more variable latency for incoming messages – a real trade-off against Task 2.4b's own 2-second fail-safe budget (Section 2.2), not a free win.

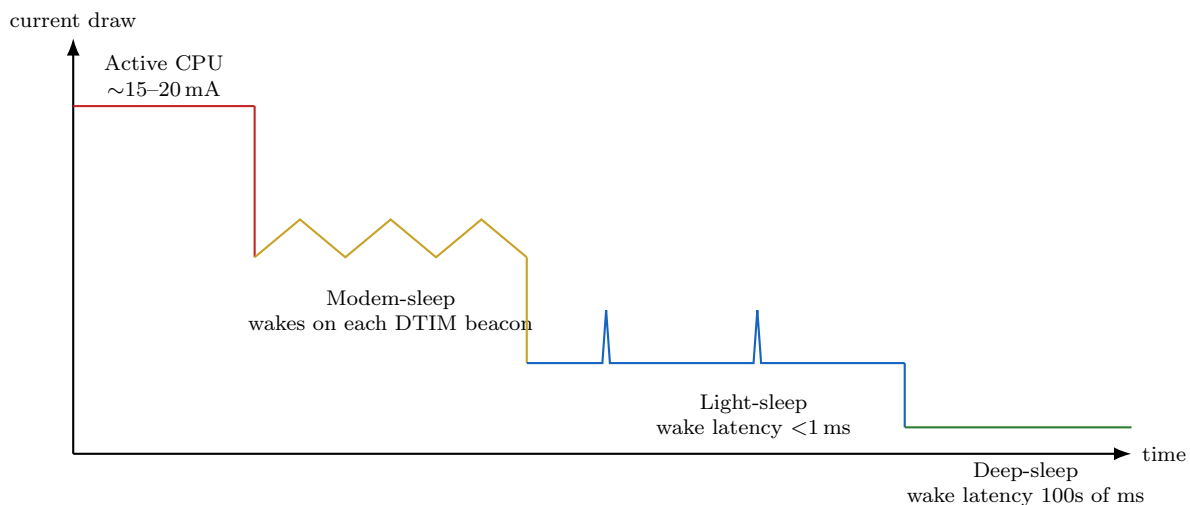


Figure 4: Approximate current-draw profile as firmware moves from an active CPU through modem-sleep and light-sleep down to deep-sleep, with each mode's typical wake-up latency noted.

3. Comprehensive Protocol Comparison: MQTT vs. ESP-NOW vs. HTTP

Table 1: Comprehensive comparison of MQTT, ESP-NOW, and HTTP, synthesized from the ESP-IDF/ESP-NOW documentation, the MQTT specification-derived sources, and published IoT protocol comparison studies (see References).

Property	MQTT	ESP-NOW	HTTP
Communication model	Publish/subscribe, broker-mediated	Direct peer-to-peer, connectionless	Client-server, request/response
OSI-style layer	Application layer, over TCP/IP	Sits directly on the 802.11 MAC layer (vendor-specific action frames); no IP layer involved	Application layer, over TCP/IP
Transport	TCP (persistent connection)	Raw 802.11 frames (connectionless, no transport-layer session)	TCP (typically one connection per request, or short-lived keep-alive)
Addressing	Logical topic strings	Physical 6-byte WiFi MAC address, hardcoded as a peer	IP address / hostname + URL path
Network requirement	Both devices need reachability to a common broker (same LAN or Internet)	Devices only need to be in radio range on the same channel; no router, AP, or Internet needed	Both devices need to be on the same reachable network; a browser/client needs the server's IP
Typical latency (small payload)	Low; a few ms to tens of ms once connected, since the TCP session is already open	Very low; typically sub-10 ms, single-hop radio with no broker relay	Higher per message; a fresh TCP handshake plus request/response round-trip unless keep-alive is used
Per-message overhead	Low; a minimal MQTT publish can be as little as ~2 bytes of protocol overhead over an already-open connection	Very low; a thin ESP-NOW header on top of the 802.11 frame, no IP/TCP headers at all	High; HTTP headers alone routinely add well over 100 bytes per request/response, even before TLS
Delivery guarantee	Configurable per-message: QoS 0 (at most once), QoS 1 (at least once), QoS 2 (exactly once)	MAC-layer send-callback only (SUCCESS/FAIL); no application-layer acknowledgement or retry built in	Reliable at the TCP layer (retransmission guaranteed), but no concept of message queuing if the client is offline
Maximum practical payload	Effectively unbounded (broker- and client-buffer limited; JSON/text payloads of many KB are routine)	250 bytes per packet on ESP-NOW v1.0; up to ~1450 bytes on v2.0	Effectively unbounded (limited by server RAM/flash and client timeout tolerance)

continued on next page

Table 1 continued from previous page

Property	MQTT	ESP-NOW	HTTP
Push vs. pull	True push – the broker delivers to subscribers the instant a message is published	True push – the sender transmits directly whenever it chooses	Pull-only in the base protocol – a client must initiate every request (WebSockets/SSE are separate extensions used to add push)
State awareness	Stateful per-connection session; optional persistent sessions and retained “last value” messages	Stateless per-packet; the application (Task 2.4a’s fail-safe) must build any notion of link health itself	Stateless by design; each request is independent unless the application layers cookies/tokens on top
Built-in liveness detection	Keep-alive pings and Last Will and Testament let the broker detect and announce a client’s disconnect	None; only application-level heuristics (e.g. a silence timeout, as built in Task 2.4a) can detect a lost peer	None natively; a client must actively re-poll to discover the server is unreachable
Security	TLS (port 8883) plus username/password or client-certificate auth at the broker; topic-level ACLs possible	Optional CCMP (AES-based) encryption per peer, capped at a limited number of simultaneously encrypted peers	TLS (HTTPS, port 443); mature, ubiquitous certificate infrastructure
Infrastructure required	A broker (self-hosted or public, e.g. Mosquitto)	None – fully peer-to-peer, no router or server of any kind	A device capable of running an HTTP server (or reaching one)
Scalability (device count)	High; a broker can fan messages out to very large numbers of subscribers	Moderate; capped by paired-peer limits (encrypted: historically 6, up to 17 configurable; unencrypted: up to 20) unless broadcast is used	High for read-heavy access, but each simultaneous client consumes server-side connection resources
Power efficiency	Good; one persistent connection amortizes overhead across many messages	Very good; no handshake, no persistent session, minimal radio-on time per message	Comparatively poor for frequent updates; each poll can imply a new handshake unless carefully kept alive
Best-suited use case (from these projects)	Task 2.4b: two boards that may not share a WiFi network, needing broker-mediated reach and easy debugging via any generic MQTT client	Task 2.4a: two boards in direct radio range needing the lowest possible latency safety link, with no router available	Task 2.3: a single device serving a human-readable, browser-based dashboard with no app or broker to install

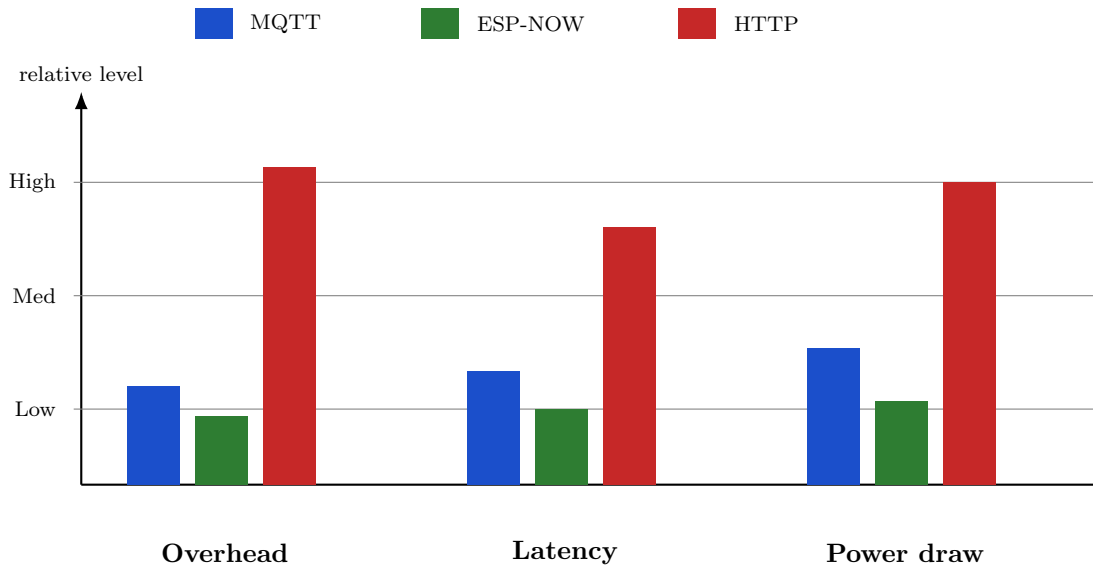


Figure 5: Qualitative summary of Table 1: relative per-message overhead, latency, and power draw across MQTT, ESP-NOW, and HTTP. Bar heights reflect the relative Low/Medium/High levels discussed in Section 3, not measured values.

4. Applying the Comparison to the Three Completed Projects

4.1 Task 2.4a (ESP-NOW) – correct choice for a direct safety link

A two-second fail-safe timeout is only meaningful if the underlying link itself contributes negligible latency compared to that budget. ESP-NOW’s single-hop, no-broker, no-handshake design means the actual transmission time is a small fraction of the 2-second window, leaving the debounce and pacing delays (500 ms and 20 ms respectively) as the dominant, deliberately-chosen sources of latency in the system – exactly the property a safety interlock needs from its transport layer: predictable, and out of the way.

4.2 Task 2.4b (MQTT) – correct choice once the boards are decoupled from a shared radio

The entire justification for re-implementing Task 2.4a over MQTT was removing the requirement that both boards share a WiFi channel and know each other’s MAC address. MQTT’s broker-mediated, IP-routable design is precisely what makes that possible, at the cost of needing a broker and a real WiFi network on both ends – a trade the project’s own report identifies correctly, and one this research confirms is architecturally unavoidable given what ESP-NOW fundamentally is (Section 2.1).

4.3 Task 2.3 (HTTP) – correct choice for a human-facing, installation-free dashboard

None of the properties that make MQTT or ESP-NOW efficient for machine-to-machine safety links – persistent connections, push delivery, minimal per-message overhead – matter for a dashboard a person opens in a browser a few times an hour. What matters instead is exactly what HTTP is built for: zero client-side installation, universal browser support, and a human-readable request/response model that made routes like `/download` trivial to implement as ordinary file responses.

5. Summary & Next Steps

This review did not change any of the three projects' original protocol choices – each was already the right fit for its constraints – but it did replace intuition with mechanism for why. ESP-NOW's 250-byte, MAC-layer, no-acknowledgement design (Section 2.1) explains why a silence-based fail-safe was always the right reliability strategy to build on top of it, rather than trusting any per-packet acknowledgement. MQTT's QoS levels, retained messages, and Last Will and Testament (Section 2.2) turn out to solve different problems than Task 2.4b's timeout-based fail-safe, rather than being redundant with it, which sharpens exactly which of Task 2.4b's own Future Improvements (LWT in particular) would meaningfully tighten the existing design instead of duplicating it. HTTP's statelessness and per-request overhead (Section 2.3) explains, at the transport level, why Task 2.3 needed an async server the moment a slow route existed. The power-efficiency research (Section 2.4) is the one topic here that changes future designs rather than just explaining past ones: none of the four builds reviewed ran on battery, so none needed modem-sleep, light-sleep, deep-sleep, or DFS – but the next battery-powered sensor node will, and the code-only techniques gathered here (RTC-memory-aware deep-sleep cycles, `esp_pm_configure()` for tasks that cannot sleep, and simply not writing to flash or transmitting more often than the data actually changes) are now a checklist to apply from the first line of firmware, not an afterthought once the board is already wired to a battery. Going forward, the three-way protocol comparison table (Section 3) is the first thing to consult when a new sensor node needs a communication protocol: direct radio range with no router available favors ESP-NOW, cross-network or Internet-scale coordination favors MQTT, and a human-facing, install-nothing interface favors HTTP – and the power checklist in Section 2.4 applies underneath whichever one is chosen.

6. References

1. Espressif Systems, *ESP-NOW – ESP-IDF Programming Guide (ESP32)*. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_now.html
2. Espressif Systems, *ESP-NOW – ESP-FAQ, Application Solutions*. https://docs.espressif.com/project_s/esp-faq/en/latest/application-solution/esp-now.html
3. Random Nerd Tutorials, *Getting Started with ESP-NOW (ESP32 with Arduino IDE)*. <https://randomnerdtutorials.com/esp-now-esp32-arduino-ide/>
4. OpenELAB Technology Ltd., *ESP-NOW: Eight Points You Must Know*. <https://openelab.io/blogs/learn/esp-now-eight-points-you-must-know>
5. Amazon Web Services, *MQTT – AWS IoT Core Developer Guide* (QoS, retained messages, Last Will and Testament, persistent sessions). <https://docs.aws.amazon.com/iot/latest/developerguide/mqtt.html>
6. EMQ Technologies, *MQTT QoS 0, 1, 2 Explained: A Quickstart Guide*. <https://www.emqx.com/en/blog/introduction-to-mqtt-qos>
7. justprotocols, *MQTT Protocol Explained: Publish-Subscribe, QoS Levels, Topics & Retained Messages*. <https://justprotocols.com/protocols/mqtt>
8. scadaprotocols, *MQTT Retained Messages Explained: Last Known Value*. <https://scadaprotocols.com/mqtt-retained-messages/>
9. IoT For All, *MQTT vs HTTP for IoT: Detailed Protocol Comparison*. <https://www.iotforall.com/mqtt-vs-http-for-iot-detailed-protocol-comparison>
10. Survey paper, *Reducing Communication Overhead in the IoT-Edge-Cloud Continuum: A Survey on Protocols and Data Reduction Strategies*, arXiv. <https://arxiv.org/pdf/2404.19492>

11. Jara Ochoa, H.J. et al., *Comparative Analysis of Power Consumption between MQTT and HTTP Protocols in an IoT Platform*, *Sensors* 23(10), 4896 (2023). <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC10224120/>
12. Insights2Techinfo, *IoT Protocols: Comparing MQTT, CoAP, and HTTP for Efficient Device Communication*. <https://insights2techinfo.com/iot-protocols-comparing-mqtt-coap-and-http-for-efficient-device-communication/>
13. Last Minute Engineers, *Insight Into ESP32 Sleep Modes & Their Power Consumption*. <https://lastminuteengineers.com/esp32-sleep-modes-power-consumption/>
14. Espressif Systems, *Power Management – ESP-IDF Programming Guide (ESP32)*. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/power_management.html
15. Espressif Systems, *Wi-Fi Performance and Power Save – ESP-IDF Programming Guide*. <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/wifi-driver/wifi-performance-and-power-save.html>
16. espboards.dev, *Deep Sleep Low Power Optimization on ESP32 Dev Boards*. <https://www.espboards.dev/blog/esp32-power-optimisation/>